Chapter 16: Query Optimization

Outline

Introduction

Transformation of Relational Expressions

Statistical Information for Cost Estimation

Cost-based Optimization

Dynamic Programming for Choosing Evaluation Plans

Nested Subqueries

Materialized Views

Advanced Topics in Query Optimization

Introduction

Parsing and translation

translate the query into its internal form. This is then translated into relational algebra.

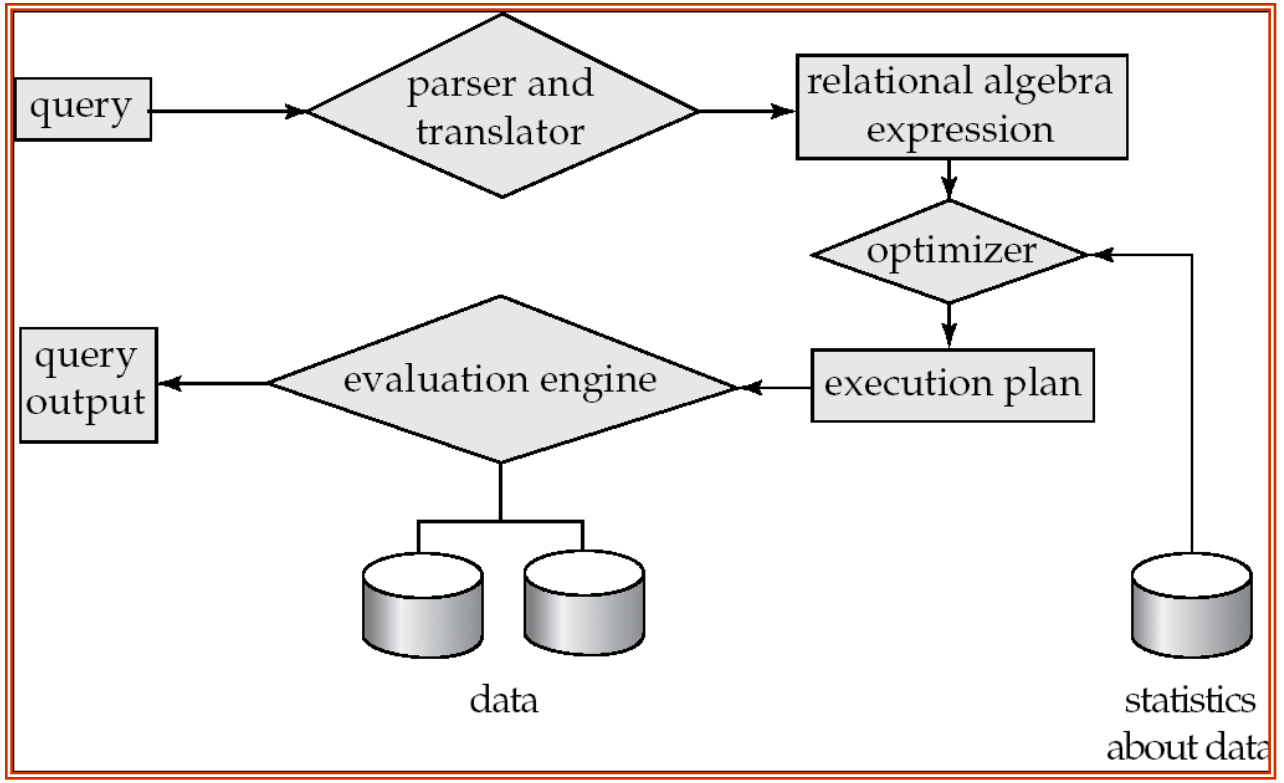
Parser checks syntax, verifies relations

Optimization

Amongst all equivalent evaluation plans choose the one with lowest cost.

Evaluation

The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.



Basic Steps in Query Processing

Introduction

Alternative ways of evaluating a given query

Equivalent expressions

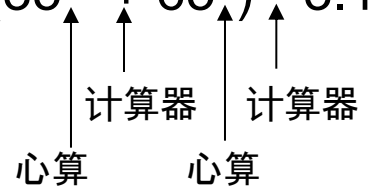
Different algorithms for each operation

An analogy example:

$3.14 * 35^2 + 3.14 * 65^2$

$= 3.14 * (35^2 + 65^2)$

$= (35^2 + 65^2) * 3.14$



← 逻辑优化，**Transformation Based Optimization**

← 物理优化，**Cost Based Optimization**

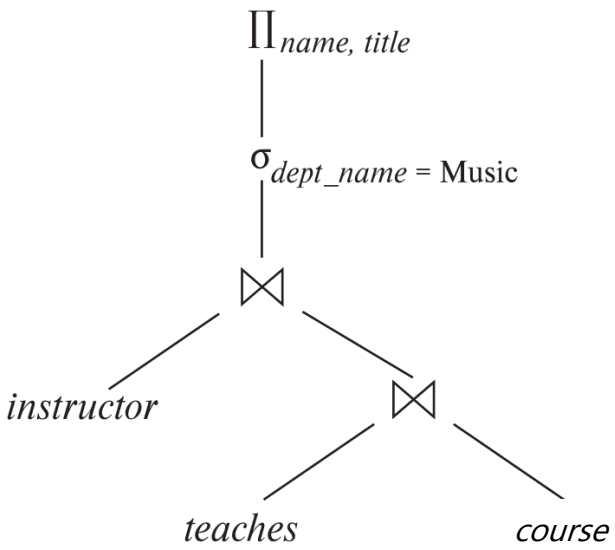
Introduction

select *name, title*

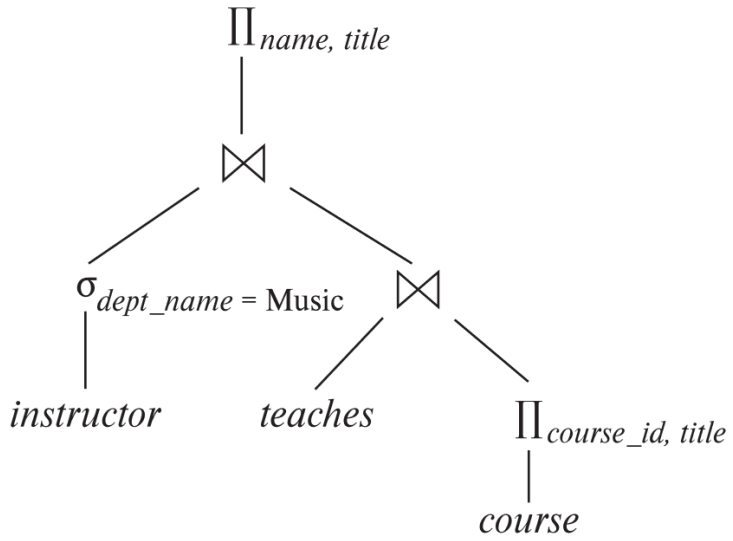
from *instructor* **natural join (***teaches* **natural join** *course*)

where *dept_name*=‘Music’ ;

$\Pi_{name,\ title}(\sigma_{dept\_name=\text{“ Music”}}(instructor \bowtie (teaches \bowtie (course))))$



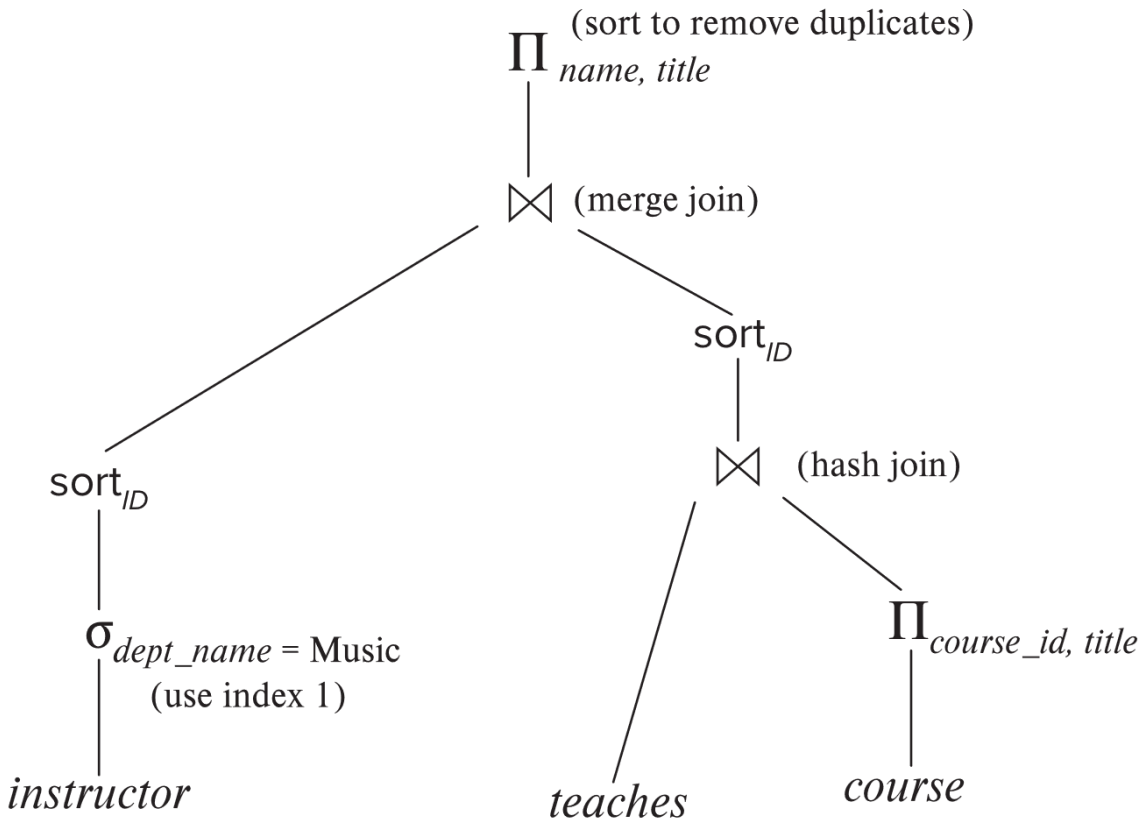
(a) Initial expression tree



(b) Transformed expression tree

Introduction (Cont.)

An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



Introduction (Cont.)

Cost difference between evaluation plans for a query can be enormous

E.g. seconds vs. days in some cases

Steps in **cost-based query optimization**

1. Generate logically equivalent expressions using **equivalence rules**
2. Annotate resultant expressions in alternative ways to get alternative **query plans**
3. Choose the cheapest plan based on **estimated cost**

Estimation of plan cost based on:

Statistical information about relations. Examples:

- number of tuples, number of distinct values for an attribute

Statistics estimation for intermediate results（Cardinality Estimation）

- to compute cost of complex expressions

Cost formulae for algorithms, computed using statistics

Viewing Query Evaluation Plans

Most database support **explain** evaluation plan for a query

Some syntax variations between databases

- PostgreSQL : **explain** <query>
- Oracle: **explain plan for** <query> followed by **select** * **from** table (*dbms_xplan.display*)
- SQL Server: **set showplan_text on**

Some databases (e.g. PostgreSQL) support **explain analyse** <query>

- Shows actual runtime statistics found by running the query, in addition to showing the plan

Some databases (e.g. PostgreSQL) show cost as *f..l*

- *f* is the cost of delivering first tuple
- *l* is cost of delivering all results

Generating Equivalent Expressions

Transformation of Relational Expressions

Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance

In SQL, inputs and outputs are multisets of tuples

> Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.

An **equivalence rule** says that expressions of two forms are equivalent

> Can replace expression of first form by second, or vice versa

Equivalence Rules

1. Conjunctive selection operations can be deconstructed（分解）into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative（可交换的）.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted（可省略的）.

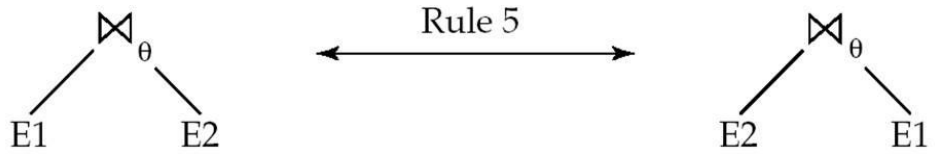
$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{Ln}(E))\ldots)) = \Pi_{L_1}(E)$$

4. Selections can be combined with Cartesian products and theta joins.
   a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$
   b. $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$

Equivalence Rules (Cont.)

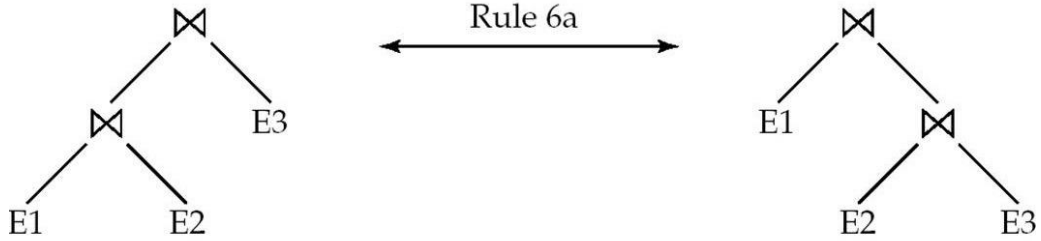
5. Theta-join operations (and natural joins) are commutative（可交换的）.

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$



6. (a) Natural join operations are associative（可结合的）:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$



(b) Theta joins are associative in the following manner:

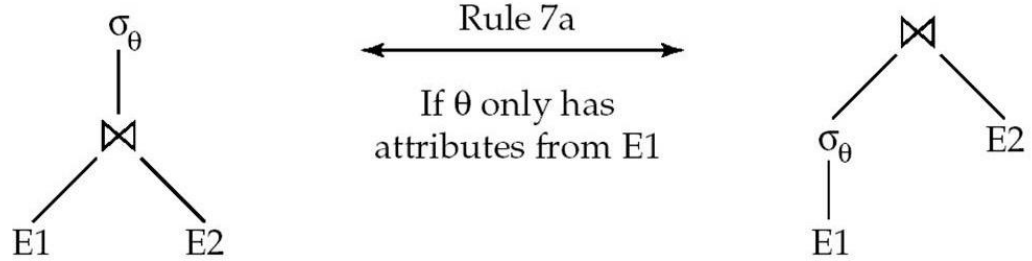
$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 = E_1 \bowtie_{\theta 1 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3.

Equivalence Rules (Cont.)

7. The selection operation **distributes（分配）** over **the theta join** operation under the following two conditions:
   (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta 0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_{\theta} E_2$$



   (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.

$$\sigma_{\theta 1 \wedge \theta 2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 1}(E_1)) \bowtie_{\theta} (\sigma_{\theta 2}(E_2))$$

Equivalence Rules (Cont.)

8. The projection operation distributes（分配）over the theta join operation as follows:

   (a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2))$$

   (b) Consider a join $E_1 \bowtie_\theta E_2$.

   Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively.

   Let L_3 be attributes of E_1 that are involved in join condition θ, but are not in $L_1 \cup L_2$, and

   let L_4 be attributes of E_2 that are involved in join condition θ, but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\Pi_{L_2 \cup L_4}(E_2)))$$

Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$
$$E_1 \cap E_2 = E_2 \cap E_1$$

(set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

Also: $$\sigma_\theta (E_1 - E_2) = \sigma_\theta(E_1) - E_2$$

and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

Equivalence Rules (Cont.)

13. Selection distributes over aggregation as below

    $\sigma_\theta({}_G\gamma_A(E)) \equiv {}_G\gamma_A(\sigma_\theta(E))$

    provided θ only involves attributes in G

14. a. Full outerjoin is commutative:

    $E_1 \mathbin{⟗} E_2 \equiv E_2 \mathbin{⟗} E_1$

    b. Left and right outerjoin are not commutative, but:

    $E_1 \mathbin{⟕} E_2 \equiv E_2 \mathbin{⟖} E_1$

15. Selection distributes over left and right outer joins as below, provided θ_1 only involves attributes of E_1

    a. $\sigma_{\theta_1}(E_1 \mathbin{⟕}_\theta E_2) \equiv (\sigma_{\theta_1}(E_1)) \mathbin{⟕}_\theta E_2$

    b. $\sigma_{\theta_1}(E_2 \mathbin{⟖}_\theta E_1) \equiv E_2 \mathbin{⟖}_\theta (\sigma_{\theta_1}(E_1))$

16. Outerjoins can be replaced by inner joins under some conditions

    a. $\sigma_{\theta_1}(E_1 \mathbin{⟕}_\theta E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_\theta E_2)$

    b. $\sigma_{\theta_1}(E_1 \mathbin{⟖}_\theta E_1) \equiv \sigma_{\theta_1}(E_1 \bowtie_\theta E_2)$

    provided θ_1 is null rejecting on E_2 :

    If θ1 has the property that it evaluates to false or unknown whenever the attributes of E2 are null.

Transformation Example: Pushing Selections

select name, title

from instructor **natural join** (teaches **natural join** course)

where dept_name='Music'

$$\Pi_{name,\ title}(\sigma_{dept\_name=\text{“Music”}}(instructor \bowtie (teaches \bowtie course)))$$

Transformation using rule 7a.

$$\Pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”}}(instructor)) \bowtie (teaches \bowtie course))$$

Performing the selection as early as possible reduces the size of the relation to be joined.

Transformation Example: Pushing Projections

Consider:

$\Pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches) \bowtie course))$

When we compute

$(\sigma_{dept\_name = \text{“Music”}}(instructor) \bowtie teaches)$

we obtain a relation whose schema is:
(*ID, name, dept_name, salary, course_id, sec_id, semester, year)*

Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

$\Pi_{name,\ title}(\Pi_{name,\ course\_id}($
$\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches)) \bowtie$
$\Pi_{course\_id,\ title}(course))))$

Performing the projection as early as possible reduces the size of the relation to be joined.

Transformation Example - Join Ordering

For all relations r_1, r_2, and r_3,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.

Transformation Example - Join Ordering

Consider the expression

$\Pi_{name,\ title}(\sigma_{dept\_name=\ \text{“Music”}}\ (instructor) \bowtie ((teaches) \bowtie \Pi_{course\_id,\ title}\ (course))\ )$

Could compute $teaches \bowtie \Pi_{course\_id,\ title}\ (course)$ first, and join result with

$\sigma_{dept\_name=\ \text{“Music”}}\ (instructor)$

but the result of the first join is likely to be a large relation.

Only a small fraction of the university's instructors are likely to be from the Music department

it is better to compute

$\sigma_{dept\_name=\ \text{“Music”}}\ (instructor) \bowtie teaches$

first.

Example with Multiple Transformations

Query: Find the names of all instructors in the Music department who have taught a course in 2017, along with the titles of the courses that they taught

$$\Pi_{name,\ title}(\sigma_{dept\_name = \text{“Music”} \wedge year = 20017} (instructor \bowtie (teaches \bowtie \Pi_{course\_id,\ title}\ (course))))$$

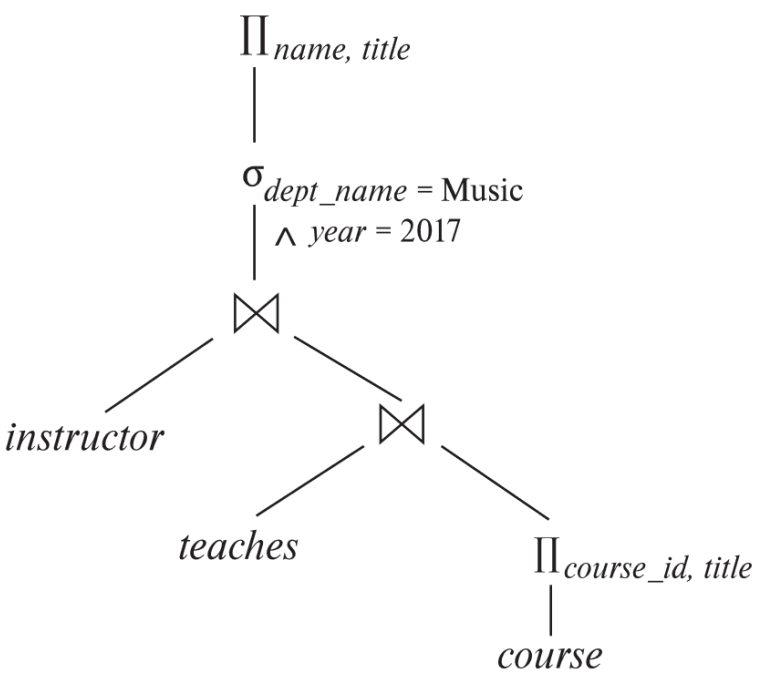
Transformation using join associatively (Rule 6a):

$$\Pi_{name,\ title}(\sigma_{dept\_name = \text{“Music”} \wedge year = 2017} ((instructor \bowtie teaches) \bowtie \Pi_{course\_id,\ title}\ (course)))$$

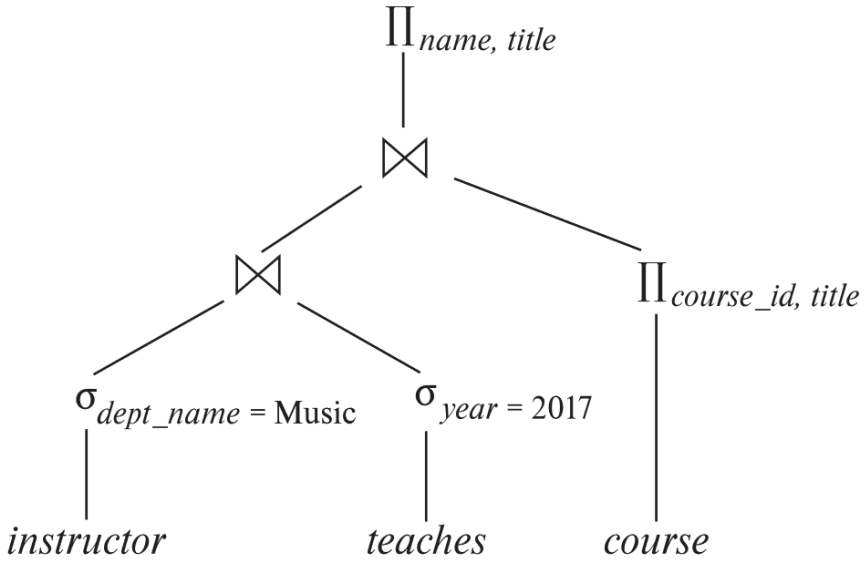
Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

$$\sigma_{dept\_name = \text{“Music”}}\ (instructor) \bowtie \sigma_{year = 2009}\ (teaches)$$

Multiple Transformations (Cont.)



(a) Initial expression tree



(b) Tree after multiple transformations

Enumeration of Equivalent Expressions

Query optimizers use equivalence rules to **systematically** generate expressions equivalent to the given expression

Can generate all equivalent expressions as follows:

Repeat

- apply all applicable equivalence rules on every subexpression of every equivalent expression found so far
- add newly generated expressions to the set of equivalent expressions

Until no new equivalent expressions are generated above

The above approach is very expensive in space and time

Two approaches

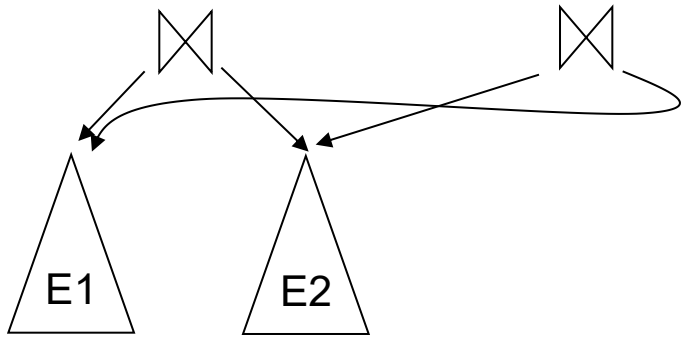
- Optimized plan generation based on transformation rules
- Special case approach for queries with only selections, projections and joins - heuristic manner .

Implementing Transformation Based Optimization

Space requirements reduced by sharing common sub-expressions:

when E1 is generated from E2 by an equivalence rule, usually only the top level of the two are different, subtrees below are the same and can be shared using pointers

- E.g. when applying join commutativity



Same sub-expression may get generated multiple times

- Detect duplicate sub-expressions and share one copy

Time requirements are reduced by not generating all expressions

Dynamic programming

- We will study only the special case of dynamic programming for join order optimization

Statistics for Cost Estimation

Cost Estimation

Cost of each operator

- Need statistics of input relations
  - E.g. number of tuples, sizes of tuples

Inputs can be results of sub-expressions

- Need to estimate statistics of expression results（cardinality estimation）
- To do so, we require additional statistics
  - E.g. number of distinct values for an attribute

Statistical Information for Cost Estimation

n_r: number of tuples in a relation *r.*

b_r: number of blocks containing tuples of *r.*

l_r: size of a tuple of *r.*

f_r: blocking factor of *r* — i.e., the number of tuples of *r* that fit into one block.

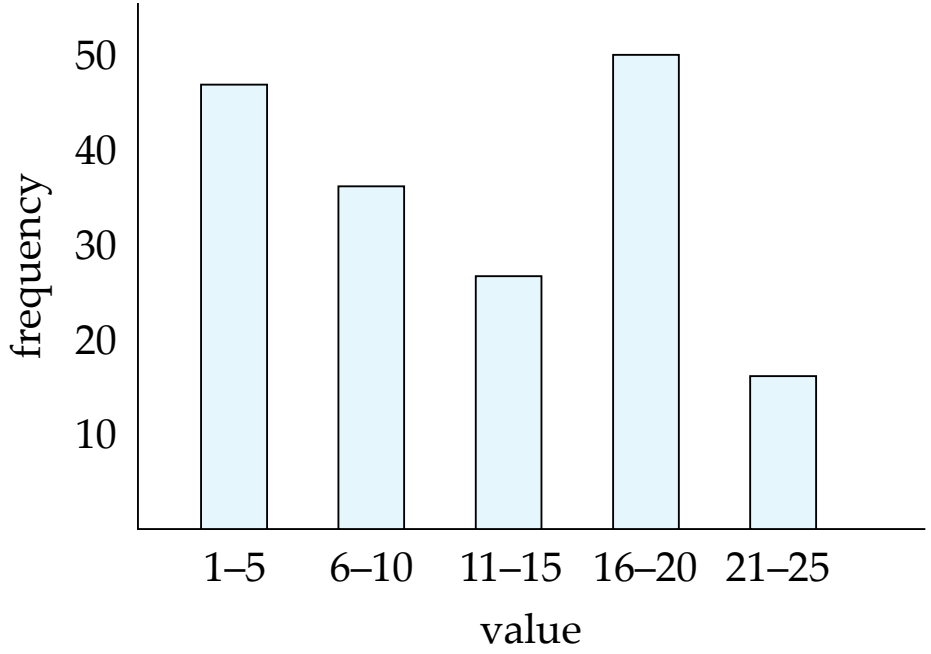
V(A, r): number of distinct values that appear in *r* for attribute *A;* same as the size of $\prod_A(r)$.

If tuples of *r* are stored together physically in a file, then:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

Histograms

Histogram on attribute *age* of relation *person*



Equi-width histograms

Equi-depth histograms

Selection Size Estimation

$\sigma_{A=v}(r)$

- $n_r / V(A,r)$: number of records that will satisfy the selection
- Equality condition on a key attribute: *size estimate* = 1

$\sigma_{A \le V}(r)$ (case of $\sigma_{A \ge V}(r)$ is symmetric)

Let c denote the estimated number of tuples satisfying the condition.

If min(A,r) and max(A,r) are available in catalog

- c = 0 if v < min(A,r)
- $c = n_r . \frac{v - \min(A,r)}{\max(A,r) - \min(A,r)}$

If histograms available, can refine above estimate

In absence of statistical information *c* is assumed to be $n_r / 2$.

Size Estimation of Complex Selections

The **selectivity（中选率）** of a condition θ_i is the probability that a tuple in the relation *r* satisfies θ_i .

If s_i is the number of satisfying tuples in *r,* the selectivity of θ_i is given by s_i / n_r.

Conjunction: $\sigma_{\theta 1 \wedge \theta 2 \wedge \ldots \wedge \theta n}(r)$. *Assuming indepdence,* estimate of tuples in the result is: $n_r * \frac{s_1 * s_2 * \ldots * s_n}{n_r^n}$

Disjunction: $\sigma_{\theta 1 \vee \theta 2 \vee \ldots \vee \theta n}(r)$. Estimated number of tuples:

$$n_r * \left[1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \ldots * (1 - \frac{s_n}{n_r})\right]$$

Negation: $\sigma_{\neg\theta}(r)$. Estimated number of tuples:

$n_r - size(\sigma_\theta(r))$

Estimation of the Size of Joins

The Cartesian product r x s contains $n_r . n_s$ tuples; each tuple occupies $s_r + s_s$ bytes.

If $R \cap S = \varnothing$, then $r \bowtie s$ is the same as r x s.

If $R \cap S$ is a key for R, then a tuple of s will join with at most one tuple from r

therefore, (the number of tuples in $r \bowtie s$) $\leq n_s$

r $n_r=3$

A	B
a1	b1
a2	b2
a3	b2

primary key

s $n_s=6$

A	C
a1	c1
a2	c2
a2	c6
a3	c3
a3	c5
a4	c4

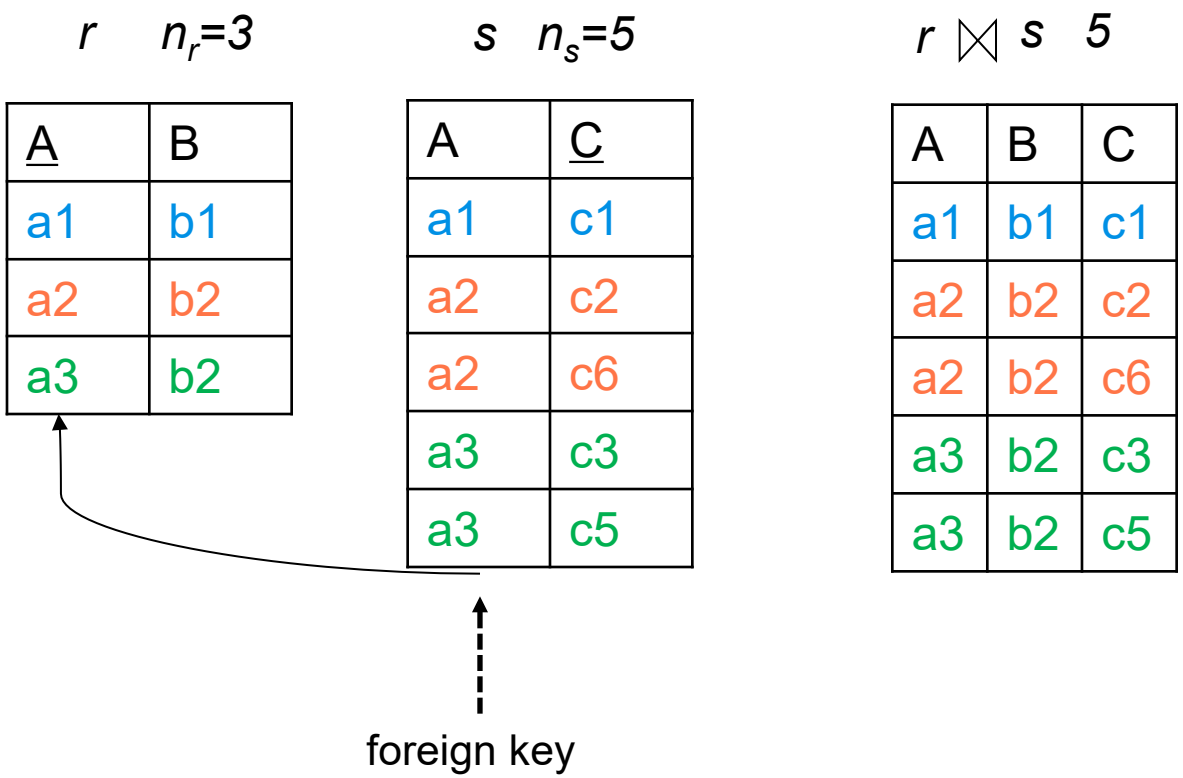
$r \bowtie s$ 5

A	B	C
a1	b1	c1
a2	b2	c2
a2	b2	c6
a3	b2	c3
a3	b2	c5

Estimation of the Size of Joins

If $R \cap S$ *in* S is a foreign key in *S* referencing *R*, then the number of tuples in $r \bowtie s$ = the number of tuples in *s.*

- The case for $R \cap S$ being a foreign key referencing *S* is symmetric.



Estimation of the Size of Joins (Cont.)

If $R \cap S = \{A\}$ is not a key for R or S.

If we assume that every tuple t in R produces tuples in R⋈S, the number of tuples in R ⋈ S is estimated to be:

$$\frac{n_r * n_s}{V(A,s)}$$

If the reverse is true, the estimate obtained will be:

$$\frac{n_r * n_s}{V(A,r)}$$

The lower of these two estimates is probably the more accurate one.

Can improve on above if histograms are available

- Use formula similar to above, for each cell of histograms on the two relations

Estimation of the Size of Joins (Cont.)

r $n_r=4$

A	B
a1	b1
a2	b2
a2	b4
a3	b3

s $n_s=6$

A	C
a1	c1
a2	c2
a2	c6
a3	c5
a3	c6
a4	c4

r $\bowtie$ ***s*** *7*

A	B	C
a1	b1	c1
a2	b2	c2
a2	b2	c6
a2	b4	c2
a2	b4	c6
a3	b3	c5
a3	b3	c6

V(A,r)=3

V(A,s)=4

$\frac{n_r * n_s}{V(A,s)}$ =24/4=6

$\frac{n_r * n_s}{V(A,r)}$ *=24/3=8*

Size Estimation for Other Operations

Projection: estimated size of $\Pi_A(r)$ = $V(A,r)$

Aggregation : estimated size of ${}_A\boldsymbol{g}_F(r)$ = $V(A,r)$

Set operations

For unions/intersections of selections on the same relation: rewrite and use size estimate for selections

- E.g. $\sigma_{\theta 1}(r) \cup \sigma_{\theta 2}(r)$ can be rewritten as $\sigma_{\theta 1 \text{ v } \theta 2}(r)$

For operations on different relations:

- estimated size of $r \cup s$ = size of r + size of s.
- estimated size of $r \cap s$ = minimum size of r and size of s.
- estimated size of $r - s$ = r.
- All the three estimates may be quite inaccurate, but provide upper bounds on the sizes.

Size Estimation (Cont.)

Outer join:

Estimated size of $r \mathbin{⟕} s$ = *size of* $r \bowtie s$ + *size of* r

- Case of right outer join is symmetric

Estimated size of $r \mathbin{⟖} s$ = *size of* $r \bowtie s$ + size of r + size of s

Estimation of Number of Distinct Values

The size estimates often depend on the number of distinct values for an attribute. Need to compute the distinct vales for intermediate results.

Selections: σ_θ (r) , estimate V(A,σ_θ (r))

If θ forces A to take a specified value: V(A,$\sigma\theta$ (r)) = 1.

- e.g., *A* = 3

If θ forces A to take on one of a specified set of values:

V(*A*,σ_θ (*r*)) = number of specified values.

- (e.g., (*A* = 1 *V A* = 3 *V A* = 4)),

If the selection condition θ is of the form *A op v*

estimated *V*(*A*,σ_θ (*r*)) = *V*(*A*.*r*) * *s*

- where *s* is the selectivity of the selection.

In all the other cases, use approximate estimate:

V(*A*,σ_θ (*r*)) = min(*V*(*A*,*r*), $n_{\sigma\theta\ (r)}$)

More accurate estimate can be got using probability theory, but this one works fine generally

Estimation of Distinct Values (Cont.)

Joins: $r \bowtie s$, *estimate* $V(A, r \bowtie s)$

If all attributes in A are from r, *the* estimated

$$V(A, r \bowtie s) = \min (V(A,r), n_{r \bowtie s})$$

If A contains attributes $A1$ from r and $A2$ from s, then estimated

$$V(A,r \bowtie s) = \min(V(A1,r)*V(A2 - A1,s), V(A1 - A2,r)*V(A2,s), n_{r \bowtie s})$$

More accurate estimate can be got using probability theory, but this one works fine generally

Estimation of Distinct Values (Cont.)

Estimation of distinct values are straightforward for projections.

They are the same in $\Pi_{A\,(r)}$ as in *r*.

The same holds for grouping attributes of aggregation.

For aggregated values

For min(*A*) and max(*A*), the number of distinct values can be estimated as min(V(*A,r*), *V*(*G,r*)) where G denotes grouping attributes

For other aggregates, assume all values are distinct, and use *V*(*G,r*)

Choice of Evaluation Plans

Cost-based Optimizer

Must consider the interaction of evaluation techniques when choosing evaluation plans

choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.

- merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
- nested-loop join may provide opportunity for pipelining

Practical query optimizers incorporate elements of the following two broad approaches:

1. Search all the plans and choose the best plan in a cost-based fashion.
2. Uses heuristics to choose a plan.

Cost-Based Join-Order Selection

Consider finding the best join-order for $r_1 \bowtie r_2 \quad \ldots r_n$.

There are $(2(n-1))!/(n-1)!$ different join orders for above expression.

with $n = 3$, the number is 12,

with $n = 4$, the number is 120,

with n = 5, the number is 1680,

with $n = 7$, the number is 665280,

with $n = 10$, the number is greater than 176 billion!

Consider joining 5 relations, suppose we want to find the best join order of the form :

12 orders

$(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$

12 orders

12*12 orders → 12+12 orders

Once we have find the best join order for the subset $\{r_1, r_2, r_3\}$, we can use that order for further join with r_4 and r_5. We need to examine only 12+12 choices.

No need to generate all the join orders. Using dynamic programming, the least-cost join order for any subset of $\{r_1, r_2, \ldots r_n\}$ is computed only once and stored for future use.

Join Order Optimization Algorithm

To find best join tree for a set of *n* relations:

To find best plan for a set *S* of *n* relations, consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of *S*.

Recursively compute costs for joining subsets of *S* to find the cost of each plan. Choose the cheapest of the $2^n - 2$ alternatives.

Base case for recursion: single relation access plan

- Apply all selections on R_i using best choice of indices on R_i

When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it

- Dynamic programming

Join Order Optimization Algorithm

```
procedure findbestplan(S)
    if (bestplan[S].cost ≠ ∞)
            return bestplan[S]
    // else bestplan[S] has not been computed earlier, compute it now
    if (S contains only 1 relation)
       set bestplan[S].plan and bestplan[S].cost based on the best way
       of accessing S  using selections on S and indices (if any) on S
     else for each non-empty subset S1 of S such that S1 ≠ S
            P1= findbestplan(S1)
            P2= findbestplan(S - S1)
            for each algorithm A for joining results of P1 and P2
               … compute plan and cost of using A (see next page) ..
                if cost < bestplan[S].cost
                        bestplan[S].cost = cost
                        bestplan[S].plan = plan;
    return bestplan[S]
```

Join Order Optimization Algorithm (cont.)

for each algorithm A for joining results of *P*1 and *P*2
// For indexed-nested loops join, the outer could be *P1* or *P2*
// Similarly for hash-join, the build relation could be *P1* or *P2*
// We assume the alternatives are considered as separate algorithms

if algorithm *A* is indexed nested loops
Let P_i and P_o denote inner and outer inputs
if P_i has a single relation r_i and r_i has an index on the join attribute

plan = "execute P_o.*plan*; join results of P_o and r_i using *A*",
with any selection conditions on P_i performed as part of the join condition

cost = P_o.*cost* + cost of *A*

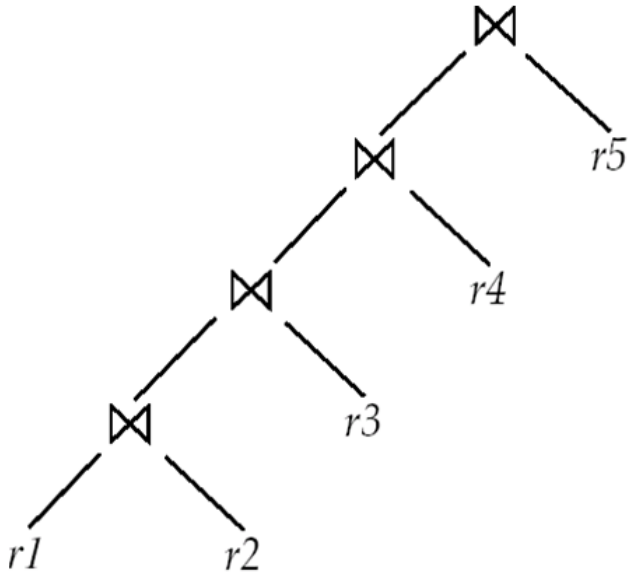
else *cost* = ∞; /* cannot use indexed nested loops join */

else
plan = "execute *P1.plan*; execute *P2.plan*;
join results of *P1* and *P2* using *A*;"
cost = *P1.cost* + *P2.cost* + cost of *A*

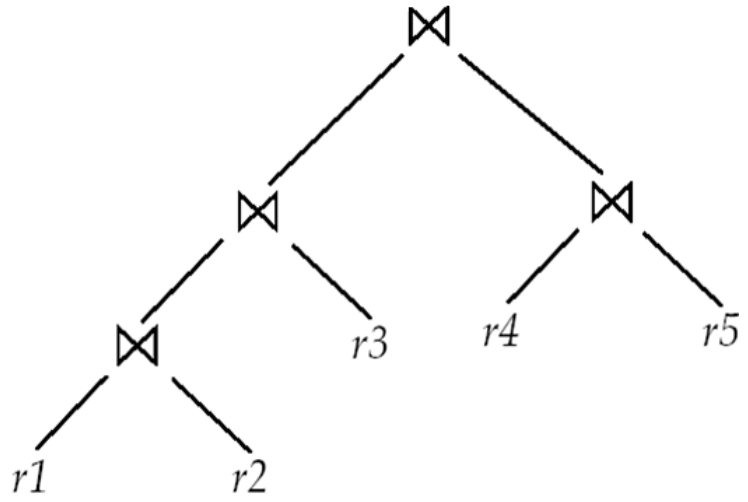
…. See previous page

Left Deep Join Trees

In **left-deep join trees**, the right-hand-side input for each join is a relation, not the result of an intermediate join.



(a) Left-deep join tree



(b) Non-left-deep join tree

Cost of Optimization

With dynamic programming

time complexity of optimization with bushy trees is $O(3^n)$.

(With n = 10, this number is 59000 instead of 176 billion!)

Space complexity is $O(2^n)$

To find best left-deep join tree for a set of n relations:

Consider n alternatives with one relation as right-hand side input and the other relations as left-hand side input.

Modify optimization algorithm:

- Replace "**for each** non-empty subset $S1$ of S such that $S1 \neq S$"
- By: **for each** relation r in S
  let S1 = S – r .

If only left-deep trees are considered,

Time complexity of finding best join order is $O(n\ 2^n)$

Space complexity remains at $O(2^n)$

Cost-based optimization is expensive, but worthwhile for queries on large datasets (typical queries have small n, generally < 10)

Heuristic Optimization(启发式优化)

Cost-based optimization is expensive, even with dynamic programming.

Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.

Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:

- Perform selection early (reduces the number of tuples)
- Perform projection early (reduces the number of attributes)
- Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.
- Perform left-deep join order
- Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Additional Optimization Techniques

Nested Subqueries

Materialized Views

Optimizing Nested Subqueries**

Nested query example:

```
select name
from instructor
where exists (select *
                from teaches
                where instructor.ID = teaches.ID and teaches.year = 2022)
```

SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return a single value or set of values

Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables（相关变量）**

Conceptually, nested subquery is executed once for each tuple in the cross-product generated by the outer level **from** clause

Such evaluation is called **correlated evaluation（相关执行）**

Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery

Optimizing Nested Subqueries (Cont.)

Correlated evaluation may be quite inefficient since

- a large number of calls may be made to the nested query
- there may be unnecessary random I/O as a result

SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques

E.g.,: earlier nested query can be rewritten as

```
select  name
from    instructor, teaches
where instructor.ID = teaches.ID and teaches.year = 2022
```

$$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID \wedge teaches.year=2022} teaches)$$

Note: the two queries generate different numbers of duplicates (why?)

- teaches can have duplicate IDs
- Can be modified to handle duplicates correctly using semijoin operators.

Optimizing Nested Subqueries (Cont.)

The **semijoin（半连接）** operator $\ltimes$ is defined as follows

If a tuple r_i appears n times in r, it appears n times in the result of $r \ltimes_\theta s$, if there is at least one tuple s_i in s *matching with* r_i.

E.g.: earlier nested query can be rewritten as

$\Pi_{name}(instructor \ltimes_{instructor.ID=teaches.ID \wedge teaches.year=2022} teaches)$

Or even as: $\Pi_{name}(instructor \ltimes_{instructor.ID=teaches.ID} (\sigma_{teaches.year=2022} teaches))$

Now the duplicate count is correct!

The above relational algebra query is also equivalent to

```
select name
from instructor
    where ID in (select teaches.ID
                 from teaches
                 where teaches.year = 2022)
```

Optimizing Nested Subqueries (Cont.)

- This could also be written using only joins (in SQL) as

```
with t1 as
    (  select distinct ID
       from teaches
       where year = 2022 )
select name
from instructor, t1
 where t1.ID = instructor.ID
```

- The query

```
select name
from instructor
where not exists (select *
                  from teaches
                  where instructor.ID = teaches.ID and teaches.year = 2022)
```

  can be rewritten using the **anti-semijoin** (anti-join) operation as $\overline{\ltimes}$

$$\Pi_{name}(instructor \,\overline{\ltimes}_{instructor.ID=teaches.ID \,\wedge\, teaches.year=2022}\, teaches)$$

Optimizing Nested Subqueries (Cont.)

In general, SQL queries of the form below can be rewritten as shown

Rewrite: **select** A
from $r_1, r_2, \ldots, r_n$
where P_1 **and exists** (**select** *
from $s_1, s_2, \ldots, s_m$
where P_2^1 **and** P_2^2)

To: $\Pi_A(\sigma_{P1}(r_1 \times r_2 \times \ldots \times r_n) \ltimes_{P2^2} \sigma_{P2^1}(s_1 \times s_2 \times \ldots \times s_m)$

P_2^1 contains predicates that do not involve any correlation variables

P_2^2 contains predicates involving correlation variables

The process of replacing a nested query by a query with a join/semijoin (possibly with a temporary relation) is called **decorrelation(去除相关）**

Decorrelation is more complicated in several cases, e.g.

- The nested subquery uses aggregation, or
- The nested subquery is a scalar subquery

Correlated evaluation used in these cases

Decorrelation (Cont.)

Decorrelation of scalar aggregate subqueries can be done using groupby/aggregation in some cases

```
select name
from instructor
where  1 < (select count(*)
                 from teaches
                 where instructor.ID = teaches.ID
                       and teaches.year = 2022)
```

$$\Pi_{name}(instructor \ltimes_{instructor.ID=TID \wedge 1 < cnt} ({}_{ID\ as\ TID}\gamma_{\mathbf{count}(*)\ as\ cnt}(\sigma_{teaches.year=2022}(teaches))))$$

Materialized Views**

A **materialized view** is a view whose contents are computed and stored.

Consider the view

```
create view department_total_salary(dept_name, total_salary) as
select dept_name, sum(salary)
from instructor
group by dept_name
```

Materializing the above view would be very useful if the total salary by department is required frequently

- Saves the effort of finding multiple tuples and adding up their amounts

Materialized View Maintenance

The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**

Materialized views can be maintained by recomputation on every update

A better option is to use **incremental view maintenance**

(增量视图维护)

Changes to database relations are used to compute changes to the materialized view, which is then updated

View maintenance can be done by

Manually defining triggers on insert, delete, and update of each relation in the view definition

Manually written code to update the view whenever database relations are updated

Periodic recomputation (e.g. nightly)

Above methods are directly supported by many database systems

- Avoids manual effort/correctness issues

Incremental View Maintenance

The changes (inserts and deletes) to a relation or expressions are referred to as its **differential（差分）**

- Set of tuples inserted to and deleted from r are denoted $\mathbf{i_r}$ and $\mathbf{d_r}$

To simplify our description, we only consider inserts and deletes

- We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple

We describe how to compute the change to the result of each relational operation, given changes to its inputs

We then outline how to handle relational algebra expressions

Join Operation

Consider the materialized view $v = r \bowtie s$ and an update to r

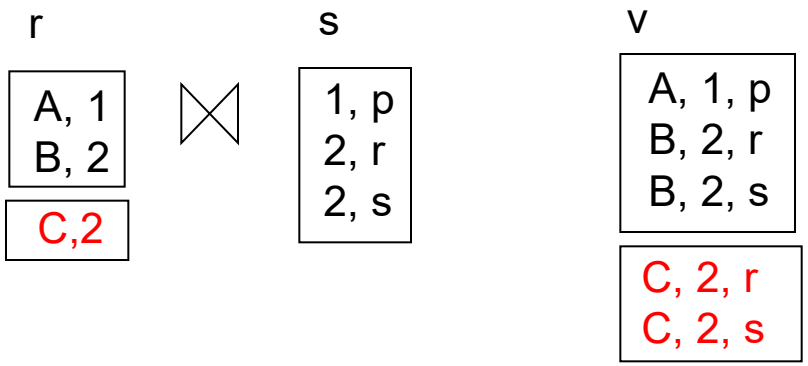
Let r^{old} and r^{new} denote the old and new states of relation r

Consider the case of an insert to r:

- We can write $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$
- And rewrite the above to $(r^{old} \bowtie s) \cup (i_r \bowtie s)$
- But $(r^{old} \bowtie s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \bowtie s$

Thus, for inserts $v^{new} = v^{old} \cup (i_r \bowtie s)$

Similarly for deletes $v^{new} = v^{old} - (d_r \bowtie s)$



Selection and Projection Operations

Selection: Consider a view $v = \sigma_\theta(r)$.

$v^{new} = v^{old} \cup \sigma_\theta(i_r)$

$v^{new} = v^{old} - \sigma_\theta(d_r)$

Projection is a more difficult operation

$R = (A,B)$, and r(R) = { (*a*,2), (*a*,3)}

$\prod_A(r)$ has a single tuple (*a*).

If we delete the tuple (*a*,2) from *r,* we should not delete the tuple (*a*) from $\prod_A(r)$, but if we then delete (*a*,3) as well, we should delete the tuple

For each tuple in a projection $\prod_A(r)$, we will keep a count of how many times it was derived

On insert of a tuple to *r*, if the resultant tuple is already in $\prod_A(r)$ we increment its count, else we add a new tuple with count = 1

On delete of a tuple from r, we decrement the count of the corresponding tuple in $\prod_A(r)$

- if the count becomes 0, we delete the tuple from $\prod_A(r)$

Aggregation Operations

count : $v = {}_{A}\mathcal{g}_{count(B)}{}^{(r)}$.

When a set of tuples i_r is inserted

- For each tuple r in i_r, if the corresponding group is already present in v, we increment its count, else we add a new tuple with count = 1

When a set of tuples d_r is deleted

- for each tuple t in i_r we look for the group $t.A$ in v, and subtract 1 from the count for the group.
  - If the count becomes 0, we delete from v the tuple for the group $t.A$

sum: $v = {}_{A}\mathcal{g}_{sum\ (B)}{}^{(r)}$

We maintain the sum in a manner similar to count, except we add/subtract the B value instead of adding/subtracting 1 for the count

Additionally we maintain the count in order to detect groups with no tuples. Such groups are deleted from v

- Cannot simply test for sum = 0 (why?)

To handle the case of **avg**, we maintain the **sum** and **count** aggregate values separately, and divide at the end

Aggregate Operations (Cont.)

min, **max**: $v = {}_{A}g_{min\ (B)}(r)$.

Handling insertions on r is straightforward.

Maintaining the aggregate values **min** and **max** on deletions may be more expensive. We have to look at the other tuples of *r* that are in the same group to find the new minimum

Other Operations

Set intersection: $v = r \cap s$

when a tuple is inserted in *r* we check if it is present in *s*, and if so we add it to *v*.

If the tuple is deleted from r, we delete it from the intersection if it is present.

Updates to *s* are symmetric

The other set operations, *union* and *set difference* are handled in a similar fashion.

Outer joins are handled in much the same way as joins but with some extra work

we leave details to you.

Handling Expressions

To handle an entire expression, we derive expressions for computing the incremental change to the result of each sub-expressions, starting from the smallest sub-expressions.

E.g. consider $E_1 \bowtie E_2$ where each of E_1 and E_2 may be a complex expression

Suppose the set of tuples to be inserted into E_1 is given by D_1

- Computed earlier, since smaller sub-expressions are handled first

Then the set of tuples to be inserted into $E_1 \bowtie E_2$ is given by $D_1 \bowtie E_2$

- This is just the usual way of maintaining joins

Query Optimization and Materialized Views

Rewriting queries to use materialized views:

A materialized view $v = r \bowtie s$ is available

A user submits a query $r \bowtie s \bowtie t$

We can rewrite the query as $v \bowtie t$

- Whether to do so depends on cost estimates for the two alternative

Replacing a use of a materialized view by the view definition:

A materialized view $v = r \bowtie s$ is available, but without any index on it

User submits a query $\sigma_{A=10}(v)$.

Suppose also that *s* has an index on the common attribute B, and r has an index on attribute A.

The best plan for this query may be to replace *v* by $r \bowtie s$, which can lead to the query plan $\sigma_{A=10}(r) \bowtie s$

Materialized View Selection

Materialized view selection:

"What is the best set of views to materialize?".

Index selection:

"what is the best set of indices to create"

closely related, to materialized view selection

- but simpler

Materialized view selection and index selection based on typical system **workload** (queries and updates)

Typical goal: minimize time to execute workload , subject to constraints on space and time taken for some critical queries/updates

One of the steps in database tuning

- more on tuning in later chapters

Commercial database systems provide tools (called "tuning assistants" or "wizards") to help the database administrator choose what indices and materialized views to create

Advanced Topics in Query Optimization

Top-K Queries

Optimization of Updates

Join Minimization

Multiquery Optimization

Parametric Query Optimization

Adaptive Query Processing

End of Chapter 16